МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 5

Варіант 6

Ліниві обчислення, генератори та itertools

Виконав: студент 2 курсу групи ФЕП-22
Мазан Давид

Перевірив: аспірант
Остріков Данило Андрійович

Львів - 2026

Мета роботи

Ознайомитися з генераторами, ітераторами та лінивими обчисленнями, навчитися працювати з нескінченними послідовностями й інструментами itertools. Реалізувати потік фінансових транзакцій, відбір додатних операцій, накопичувальний баланс і перші N ковзних сум за варіантом 6.

Теоретичні відомості

Ітерований об’єкт дозволяє отримати ітератор, а ітератор зберігає позицію та повертає наступне значення. Генераторна функція з yield створює ітератор, що обчислює значення лише при запиті. Це знижує використання пам’яті й дає змогу обробляти нескінченні джерела, якщо споживання обмежене take або islice.

Модуль itertools надає інструменти для побудови потоків. islice обмежує кількість прочитаних елементів, count створює нескінченну послідовність цілих чисел, а accumulate повертає проміжні результати згортання, наприклад поточний баланс. Ковзне вікно реалізовано через deque(maxlen=k), зберігаючи лише k останніх значень.

Постановка завдання

Варіант 6: обробити лінивий потік сум фінансових транзакцій, залишити додатні операції, побудувати накопичувальний баланс і сформувати перші N ковзних сум. Загальні вимоги роботи також передбачають нескінченний генератор Fibonacci, функції take/drop/sliding_window, приклад accumulate та тест, що підтверджує ліниве споживання джерела.

Хід роботи

1. Створено проєкт lab05 зі streams.py, core.py, main.py, конфігураціями, тестами, ASSIGNMENT.md і папкою report. Результат демонстраційного запуску збережено у demo_output.txt.

2. У streams.py реалізовано генератори fib_stream і naturals, утиліти take/drop на основі islice, sliding_window на основі deque, moving_average та лініві обгортки над takewhile, dropwhile, cycle і repeat.

3. У core.py генераторний вираз відбирає лише додатні суми. accumulate створює послідовність балансів. first_sliding_sums поєднує ковзні вікна з islice, тому повертає лише запитані N сум.

4. Тести перевіряють Fibonacci, take/drop, вікна й середні, префіксні суми/добутки, обгортки itertools, крайні випадки та лінивість джерела.

Структура проєкту

```
lab05/
├── report/ (lab5.docx, lab5.pdf)
├── tests/ (fib_stream, take_drop, sliding_window, accumulate, laziness, itertools_helpers)
├── ASSIGNMENT.md
├── core.py
├── demo_output.txt
├── main.py
├── mypy.ini
├── pyproject.toml
├── README.md
├── requirements.txt
├── streams.py
└── test_results.txt
```

Результати виконання

Для набору [-12, 100, 45.5, 0, -8, 25, 60, 10] відібрано додатні транзакції [100, 45.5, 25, 60, 10]. Баланс: [100, 145.5, 170.5, 230.5, 240.5]. Перші три ковзні суми з вікном 3: [170.5, 130.5, 95.0].

```
=== Ліниві потоки: фінансові транзакції, варіант 6 ===
Додатні транзакції: [100.0, 45.5, 25.0, 60.0, 10.0]
Накопичувальний баланс: [100.0, 145.5, 170.5, 230.5, 240.5]
Перші 3 ковзні суми (вікно 3): [170.5, 130.5, 95.0]
Префіксні добутки: [1, 2, 6, 24]
```

Команду pytest -q виконано: 14 passed in 0.08s. У тесті take(3) запросив лише три елементи, а формування двох вікон розміру 2 спожило рівно три транзакції. Перевірки mypy і ruff пройшли без зауважень; код відформатований black.

Висновок

Створено ліниві генератори й конвеєр аналізу транзакцій. Він відбирає додатні операції, формує поточний баланс і перші ковзні суми, не читаючи джерело далі за потрібну кількість значень. Усі 14 тестів підтвердили правильність результатів і лінивість обчислень.